CVL: A Causal Visual Language Isomorphic to Minimal Theories

General Proposal for the Diagrammatic Layer of the Casebook and of Causal-Architect

Gerd Graßhoff iD
Humboldt-Universität zu Berlin, Forschungsinstitut des Deutschen Museums
gerd.grasshoff@hu-berlin.de

September 29, 2026

Abstract

This proposal defines CVL, a causal visual language whose diagrams are isomorphic — element by element, not merely suggestive — to the formal representations of the regularity-theoretic inference apparatus (models.qmd): atomic and complex minimal theories in the tradition of Graßhoff and May. Because the mapping is a bijection on well-formed representations, diagrams are not illustrations of the formalism but an alternative syntax for it: every visual operation corresponds to a proof rule, every proof rule has a visual counterpart, and conversion in both directions is mechanical. The language displays causal asymmetry and hypergraph structure natively, supports collapsing and expanding of large graphs with exact formal semantics, and comes with converters between formal theory and visual display. A canonical-diagram rule is adopted for the casebook: every case receives a CVL graph as part of its inference panel.

keywords causal hypergraphs · diagrammatic reasoning · minimal theories · visual language · regularity theory

Table of contents

1 Motivation: why a visual layer, and why isomorphic **2**
2 The language **2**
2.1 How to read a CVL diagram 2
2.2 The graphemes 3
2.3 Elementary set and admissible composites 5
2.4 Display of causal asymmetry 5
2.5 Token layer 5
2.6 Derivation as graph transformation 5
3 Collapse and expand **6**
4 The isomorphism, stated precisely **7**
5 Tool review and rendering stack **7**
6 Serialisation and converters **9**
7 The canonical-diagram rule **9**
8 Relation to existing notations **10**
References **10**

> **Note**
>
> **Causal Regularity Bundle — visual language.** This document is an auxiliary component of the *Causal Regularity Bundle*. It defines CVL, the causal visual language whose diagrams are isomorphic to the formal representations of `models.qmd` and are used throughout the casebook and the foundational paper (`arxiv-causal-regularity.qmd`). Bundle concept and manifest: `bundle-hybrid-publication.qmd`, `bundle.yaml`.

1 Motivation: why a visual layer, and why isomorphic

Case stories in the causation literature have always been accompanied by pictures — neuron diagrams (Baumgartner 2013), grey/white node graphs (Andreas and Günther 2024), causal hypergraphs (Baumgartner and Falk 2023). But these pictures are, in most of the literature, informal aids: nothing guarantees that the picture and the formula agree, and nothing can be *computed* from the picture. The casebook's inference panels create the opportunity to do better. Since every case is already normalised into a factor set and a Boolean dependency structure, a diagram can be generated *from* the formal representation, and edited diagrams can be parsed *back into* it. Two requirements follow.

Isomorphism. The visual language must have exactly the expressive resources of the formal language — conjunction, disjunction, negation, biconditional orientation, and nothing else. Any diagram feature without a formal counterpart invites misreading; any formal feature without a visual counterpart forces annotation in prose. The standard directed-graph vocabulary (plain arrows between nodes) fails this test: an arrow $A \to E$ cannot distinguish "A is sufficient for E" from "A is an INUS condition of E within some conjunction", which is precisely the distinction regularity theory lives on. The unit of causal structure is not the arrow but the *minimally sufficient condition as a whole* — a hyperedge — and the *set of alternative hyperedges* converging on an outcome — a minimally necessary disjunction. DAG arrows are the wrong primitive; hyperedges are the right one. CVL adopts the hypergraph semantics of Baumgartner and Falk (2023) and renders it in the arc notation of the teaching book of Baumgartner and Graßhoff (2004): one arrow per conjunct, tied by an arc.

Facilitation. Once the diagram is isomorphic to the formula, the diagram can carry the *derivation*, not only the model: redundancy eliminations become visible deletions, active clusters become highlighted chains, and the staged proofs of the casebook can be read as sequences of graph transformations. For beginners this is the shortest way into the theory — the hooters solution is one deleted hyperedge.

2 The language

2.1 How to read a CVL diagram

A CVL diagram is read in four steps; nothing else is needed.

Step 1 — find the effect. Arrows point from causes to effects, left to right. The rightmost factor node with incoming arrowheads is the outcome under analysis.

Step 2 — look for arcs. Every factor sends its own arrow to the effect it helps bring about. Where several arrows are *tied together by an arc drawn across their shafts* near the effect, those arrows form

ONE complex cause: its parts are individually insufficient and only jointly sufficient ("struck *and* dry *and* oxygen"). The arc is the conjunction bracket of the diagram language — the graphical counterpart of "$*$".

Step 3 — group the clusters. Arrows *not* tied by an arc are alternative causes: each untied arrow (or group of arrows tied by one arc) is one complete cluster that brings the effect about on its own ("the match lights if struck *or* if exposed to flame"). **This is the crucial visual distinction: arc = AND (parts of one cause); no arc = OR (alternative causes).** On any occasion of the effect, at least one complete cluster is instantiated — the disjunction of minimally sufficient conditions. The notation is the arc convention of the teaching book of Baumgartner and Graßhoff (2004), carried over to the hypergraphs of Baumgartner and Falk (2023).

Step 4 — check the marks. A small square carrying '¬' sits on a shaft just clear of the source factor: the *absence* of that factor is the working part ("…and *not* watered"). If the outcome node itself is labelled "not E", the whole theory describes the clusters for E's *failure* to occur. Grey filling means the factor occurred in the story; white with a dashed border means it did not. A **hollow arrowhead** marks a *causal regularity* — a claim about event types, holding whenever those types are instantiated. A **filled arrowhead** marks an *instantiated regularity* — the cluster that was actually active in the concrete case; inactive clusters stay hollow.

2.2 The graphemes

Every symbol of the language, rendered by the same engine that renders the casebook (click and hover work here exactly as there), each with its formal counterpart and its causal reading. A diagram feature that does not appear in this list means nothing — there are no decorative elements.

G1 — Alternative causes (OR). Two arrows into E, *no arc*. Formal counterpart: $A + B \Rightarrow E$, a minimally necessary disjunction with two disjuncts. Causal reading: A and B are *alternative* causes; each alone brings about E; on any occasion of E at least one of them is present.

G2 — One complex cause (AND). The same two arrows into E, now *tied by the arc* drawn across their shafts near E. Formal counterpart: $A * B \Rightarrow E$, one minimally sufficient condition with two conjuncts. Causal reading: A and B are *parts of one cause*; neither suffices alone; jointly they bring about E. Contrast with G1: the only difference between the two diagrams is the arc — and that is the whole difference between "and" and "or".

Where the arc begins and ends. The arc is drawn across the shafts it ties, and it stops on the outermost two of them: it begins on the shaft of the first conjunct and ends on the shaft of the last, reaching past neither. This is not a matter of taste. An arc is read by which shafts it crosses, so an arc extending beyond its outermost conjunct points at a shaft it does not tie, and at a glance the diagram states a larger cluster than the formula does. Every renderer of this language holds to it, and the arc's endpoints therefore also mark, exactly, which arrows belong to the minimally sufficient condition.

G3 — Negation port (absence as cause-part). A small square containing '¬' sits on the shaft near F and marks that the working conjunct is f, the *absence* of F. Formal counterpart: $C * f \Rightarrow E$. Causal

reading: E comes about when C is present *and* F fails to occur — the grapheme of omissions and preventers. The mark lives on the shaft alone; no diamond is drawn.

G4 — Clusters for an absence. When the *outcome node itself* carries the label 'not E', the diagram describes the clusters for E's failure to occur. Formal counterpart: $F + d \Rightarrow \bar{E}$. Causal reading: these are the factors causally relevant to E's absence (survival, prevention); used in the bogus-prevention family.

G5 — Token layer. Grey fill: the factor is instantiated in the story's concrete episode. White with dashed border: it is absent. Plain white with solid border: the diagram is type-level only, no token question is asked. Formal counterpart: the row of the coincidence table marked '◀'.

G6 — Causal level (regularity vs. instantiated). Every diagram asserts its level through the arrowhead fill.

Hollow arrowhead — a **causal regularity**: a claim about event types, asserting that whenever those types are instantiated the effect type follows. The factors are repeatable conditions, not particular episodes. Formal counterpart: a minimal theory $\Sigma_1 + \ldots + \Sigma_k \Rightarrow E$ over event-type variables.

Filled arrowhead — an **instantiated regularity**: the same minimal theory applied to a concrete episode, with the cluster that was actually active filled in and inactive clusters left hollow. This is Baumgartner's *active causal route*, rendered here as the **active cluster** under the one-concept-one-word rule. Node shading (G5) provides the token evidence; the filled head certifies which cluster satisfied the evidence.

The decision is made **per cluster**, so in an instance diagram the active cluster has filled heads and non-active clusters retain hollow heads. A regularity diagram has no filled heads at all.

> **Note**
>
> **UNORIENTABLE.** When the data support only a single cluster into an outcome — a biconditional with $k = 1$, violating the minimal-complexity requirement (CPLX) — the engine marks the result `UNORIENTABLE`. The diagram is still drawn, but the prose of the casebook panel always flags the idealisation. No special arrowhead encodes this status: the flag lives in the text and the panel's verdict badge.

G7 — Module box (complex minimal theory). A rectangle enclosing a sub-hypergraph with a designated interface (its outermost outcome and exogenous nodes); carries the collapse/expand semantics of Section 3 and is rendered in the viewer as a semantic rewrite (the *collapse C / expand C* buttons of the chain case), not as a mere visual fold.

Two further display rules complete the language: one node per factor (literals do not get separate nodes — negation is boxed on the shaft, G3), and no separate disjunction symbol (the complete in-bundle of a node *is* the minimally necessary disjunction, G1).

2.3 Elementary set and admissible composites

The six graphemes above constitute the **elementary set**: every well-formed CVL diagram is built from them and nothing else. The diagrams of the casebook are **admissible composites** — legal combinations of the elementary marks. The distinction is pedagogically useful: a newcomer needs only the elementary set to read any casebook diagram; the composites arise by combining those marks according to the isomorphism of Section 4.

2.4 Display of causal asymmetry

Causal direction is displayed twice, corresponding to its two formal grounds:

Arrowheads point from conditions to outcomes only; there are no bidirectional or reversed edges. The setting semantics of the Andreas–Günther models (forward-directed inferability) and the ⇒ orientation of minimal theories coincide in this convention.

Fan-in makes the asymmetry of determination visible: a well-formed outcome node has at least two incoming hyperedges — the (CPLX) requirement. The eye can verify directedness structurally: alternative clusters converge on effects, they do not diverge from them. Cases where only one cluster enters an outcome are marked `UNORIENTABLE` in the engine and flagged in the casebook prose; no grapheme marks this status — the flag is textual, not visual.

2.5 Token layer

Type-level structure and token-level instantiation are separated into layers, following the neuron-diagram convention:

- instantiated factor: shaded (grey) node;
- absent factor: white node;
- active cluster (rule (AR)): filled arrowhead (G6);
- inactive or preempted clusters: hollow arrowhead;
- eliminated elements (during derivations): dashed and greyed out.

The token row marked '◀' in a panel's coincidence table and the shading of the diagram are two views of the same assignment; the converters keep them synchronised.

2.6 Derivation as graph transformation

Every elimination rule of the proof calculus (models.qmd, Stage III) is a visual deletion, every check a visual inspection:

Rule	Visual counterpart
(MS)	delete one arrow from a cluster, shortening the arc (idle conjunct falls off)
(MN)	delete an entire cluster — an untied arrow, or an arc with all its arrows (free-riding alternative cause falls off)
(SM)	delete an entire in-bundle (structurally redundant biconditional)
(PERM)	re-run the display over an expanded factor set; spurious elements that were deleted must stay deleted

Rule	Visual counterpart
(AR)	fill the heads of the clusters all of whose literals are satisfied in the token layer
(MARK)	insert a marking-factor node splitting a process stage
(MAX)/(SET)	shade the background F'; run forward along arrows only
(LREL)/(PATH)	fade regularities removed from L'; descendants' in-bundles may not be faded
(DEV)	badge off-path/root nodes with their deviancy status (default / deviant)
(TYP)	grey out truncated rows of the companion table and re-run (MS)/(MN)

A staged derivation is rendered as a *filmstrip*: the initial graph, then one frame per elimination, ending with the minimal theory. In the web presentation the frames can be stepped through; in print they appear as small multiples. The hooters filmstrip has exactly three frames: full candidate structure; (MN) deletes the cluster $M * \neg D$; result $T + B \Rightarrow L$.

3 Collapse and expand

Large structures require abstraction, but abstraction must not be informal. CVL's module boxes have exact semantics, taken from the two composition results of the formal layer.

Chain collapse. A module enclosing $\Phi_1 : A + B \Rightarrow C$ and $\Phi_2 : C + D \Rightarrow E$ may be collapsed to the single displayed theory $A + B + D \Rightarrow E$ — the substitution equivalence established in the chain case of the casebook ($\Phi_1 * \Phi_2 \dashv\vdash \Phi_1 * \Phi_3$). Collapsing hides the intermediate factor C; the collapsed view is *lossy in a documented way*: it cannot distinguish the chain from the corresponding common-cause structure (the functional ambiguity). CVL therefore badges every collapsed module with the count of hidden factors, and expansion restores the inner structure exactly. Collapse is safe for navigation; verdicts about hidden factors require expansion — a constraint the converters enforce by refusing to answer relevance queries about hidden factors from a collapsed serialisation.

Background collapse. Factors in $\mathcal{F}$ that take the same value in every displayed configuration — factors with no variation in the displayed slice of the field — are not drawn as nodes; the module box carries them as a legend annotation. This is loss-free for the displayed slice, since a factor constant across all displayed cases supports no difference-making claims within that slice. Note: this is a display simplification, not a claim that the factor is a background condition (which would mean it is outside $\mathcal{F}$ entirely; see *background conditions*, I.1 in the register).

The two operations give the language scalability: a `ComplexMinimalTheory` with dozens of factors is displayed as a small graph of module boxes, each expandable to its atomic minimal theories, recursively, down to single hyperedges.

4 The isomorphism, stated precisely

Let a *well-formed CVL graph* be a finite set of factor nodes, arrows, negation ports, conjunction arcs, and module boxes obeying: every arrow runs from a factor to a factor; every arc ties ≥ 2 arrows with a common head; every arrow is tied by at most one arc; in-bundles are non-empty; module boxes nest properly. Define the reading map ρ:

- each group of arrows tied by one arc, with tails $\ell_1, ..., \ell_n$ (literal = tail node, negated if its shaft carries a '¬' box) and common head E reads as the minimally sufficient condition $\ell_1 * \cdots * \ell_n$ of E; an untied arrow reads as a single-literal condition;
- each node's in-bundle reads as the atomic minimal theory $\Sigma_1 + ... + \Sigma_k \Rightarrow E$;
- the whole graph reads as the conjunction of its atomic minimal theories — a complex minimal theory; module boxes read as parenthesized subconjunctions.

ρ is a bijection between well-formed CVL graphs (up to layout) and complex minimal theories in canonical DNF form (up to reordering of conjuncts and disjuncts): distinct graphs read as distinct theories, and every theory has a graph — constructed by introducing one junction per non-degenerate minimally sufficient condition. Layout (positions, colours, curvature) is explicitly *not* part of the language; two diagrams differing only in layout are the same CVL graph. This is the sense in which the visual language is a syntax, not an illustration: the casebook's derivations could be carried out entirely on diagrams, with ρ transporting soundness.

Two caveats delimit the isomorphism. First, coincidence tables (Stage II evidence) are not part of the graph language; graphs display *models*, tables display *evidence* — the panel keeps both. Second, the epistemic apparatus RT-B is only partially covered: prioritised belief bases can be displayed (generalisations as ticket-styled hyperedges, entrenchment as layering), but contraction dynamics are table-like and remain textual.

5 Tool review and rendering stack

The rendering layer was selected after a review of the current (2026) visualisation landscape. Requirements, in order of priority: faithful hyperedge display (junction nodes, custom arrowheads for the '⋄' port); programmatic graph rewriting (for semantic collapse/expand and derivation steps); rich interaction (click, hover, popups) with small code overhead; high-quality automatic layout for layered, DAG-like structures; embeddability in Quarto HTML without a build step; and open licensing. Graph sizes in this project are small (5–15 nodes per case; up to a few hundred for real `ComplexMinimalTheory` structures), so raw rendering throughput is a non-issue and the decision turns on expressiveness and interaction. A key requirement is the arrowhead distinction (G6): hollow for causal regularities, filled for the active cluster in an instantiated regularity.

General-purpose JS graph libraries. Cytoscape.js (MIT; ~500K weekly downloads) is the mature standard for analysis-plus-visualisation: a full stylesheet system, custom arrow shapes at both edge ends, compound nodes, an event model for click/hover on nodes and edges, PNG/SVG export, and a large extension ecosystem (comparison; Linkurious survey). vis-network offers good out-of-box interactivity but a much weaker styling model and no per-end arrow control. Sigma.js (WebGL)

targets 100K+-node graphs — capability we do not need — at the cost of expressiveness (Memgraph review). G6 (AntV) is expressive and performant with built-in collapse behaviours, but its idiom and documentation are Chinese-ecosystem-first and its API has churned across major versions. D3 gives total control at total cost — everything hand-built. Commercial options (yFiles, GoJS, Ogma/Linkurious) offer the most polished layouts and interaction toolkits but are closed-source and license-encumbered — unsuitable for a reproducible scholarly artefact.

Layout engines. Automatic layout quality is the main visible difference between amateur and professional graph rendering. The Eclipse Layout Kernel (ELK) provides the reference implementation of Sugiyama-style layered layout — crossing minimisation, layer assignment, edge routing — compiled to JavaScript as `elkjs`, with a maintained Cytoscape adapter (cytoscape-elk). Layered layout is exactly right for CVL: causes left, effects right, junctions between layers, so the visual flow *is* the causal direction. Dagre is the lighter alternative with noticeably weaker crossing minimisation; fcose is the best force-directed option for compound graphs but force-directed layouts obscure directionality — wrong for a language whose first display obligation is asymmetry.

Interaction utilities. For expand/collapse, the cytoscape.js-expand-collapse extension folds compound nodes *visually*; CVL instead implements collapse as a **semantic graph rewrite** (substitution of the collapsed factor per Section 3), toggling between two formally derived graphs — exactness over convenience, and one dependency fewer. Popups are implemented as a lightweight overlay rather than via tippy/popper: the information shown (factor readings, MSC formulas, licence notes) comes from the CVL term itself, so no external tooltip engine is needed.

Chosen stack. A bespoke SVG renderer over elkjs. The decisive consideration outranks any library feature list: *no off-the-shelf graph library draws the conjunction arc* — the tie across the shafts of several arrows that is the defining grapheme of the textbook notation. Cytoscape.js (used in an intermediate draft with a junction-disc encoding) was therefore dropped: its edges are styleable but its canvas is closed to inter-edge decorations. The current renderer (`casebook-sources/cvl-viewer.js`) keeps ELK's layered layout for the professional left-to-right flow and paints nodes, arrows, negation marks, and conjunction arcs directly as SVG — every grapheme under full control, including the hollow/filled arrowhead distinction of G6, interaction via native DOM events, and a built-in longest-path fallback layout so the HTML remains functional offline. Mermaid, used in the first draft, survives only as the plain-text round-trip serialisation inside `cvl.py`. For print, the viewer's PNG export renders at publication resolution from the same SVG; a TikZ emitter fed from the same CVL terms is the designated LaTeX path.

The interactive viewer. The reference implementation (`casebook-sources/cvl-viewer.js`, generated assets `cvl-cases.json` and `cvl-viewer-include.html`, standalone demo `cvl-viewer.html`) renders every panel graph of the casebook with: click on a factor node — popup with its natural-language reading and token state; click on an arrow or on a conjunction arc — popup with the minimally sufficient condition it belongs to, whether it is a conjunct of one complex cause or an alternative cluster, and its licence status (e.g. "deleted by (MN)"); hover — highlights the entire minimally sufficient condition (all arc-tied arrows together), making the MSC the unit of perception as it is the unit of

theory; toolbar buttons — semantic collapse/expand (chain case), derivation stepping (hooters: *apply (MN)* deletes the free-rider condition, arrows and arc together, live), and PNG export of the current view. All graph data is generated from the formal models by `casebook-sources/gen_viewer.py`; no diagram is hand-drawn.

6 Serialisation and converters

The exchange format is deliberately dual: a formal side (the `causal-architect` model classes) and a visual side (any renderer that can draw labelled hypergraphs).

Formal → visual. `mt_to_cvl(theory)` emits a CVL term: a JSON object with `factors`, `edges` (each edge a list of signed tails and one head, with optional head negation), `modules`, and an optional `token` assignment. Renderers consume CVL terms. The reference renderer (`term_to_viz` in `gen_viewer.py`) emits the viewer's graph spec — one arrow per conjunct, arc groups for every minimally sufficient condition with ≥ 2 conjuncts, negation marks as a small '¬' box on the shaft near the source (G3), outcome negation in the node label (G4), hollow heads for causal regularities and filled heads for instantiated regularity active clusters (G6), token shading on nodes — laid out by ELK's layered algorithm and painted and made interactive by the SVG renderer `cvl-viewer.js` (Section 5). A text renderer emits Mermaid, retained solely as a human-readable serialisation with a tested inverse parser; a TikZ/SVG emitter with proper '•' and '◇' glyphs is the print target.

Visual → formal. `cvl_to_mt(term)` parses a CVL term back into the model classes; `mermaid_to_cvl(text)` parses the Mermaid subset emitted by the reference renderer, so that hand-edited diagrams round-trip. The round-trip laws are the correctness criterion of the toolchain:

```
cvl_to_mt(mt_to_cvl(T)) == T          for every complex minimal theory T
mermaid_to_cvl(cvl_to_mermaid(G)) == G  for every CVL term G
```

Both laws are enforced by tests over all casebook structures (`casebook-sources/cvl.py`; the test run is part of the casebook's verification suite). Because the mapping is bijective, the toolchain also *facilitates problem representation*: a user may sketch the structure of a new case diagrammatically, parse it, and obtain coincidence-table generation, minimisation, and the full proof calculus for free — Stage I normalisation by drawing.

7 The canonical-diagram rule

Adopted for the casebook and all future case analyses:

> **Rule (Stage G).** Every case receives a CVL graph as part of its inference panel, placed between Stage I (normalisation) and Stage II (coincidence table). The graph displays the structure under test with the token layer of the case; factors eliminated in the derivation are the ones whose visual elements the derivation deletes. Where a case involves a modelling dispute (switch, boulder), each disputed model gets its own graph — the dispute is then *visible* as a difference in graphs, not only in verdicts.

The rule has a quality-control function beyond presentation: drawing the graph forces normalisation to be complete (every mentioned mechanism either appears as a hyperedge or is explicitly relegated to the field), and the graph's fan-in displays at a glance whether (CPLX) is satisfiable — single-in-bundle nodes signal idealisations that the prose must acknowledge, as the casebook's subcause and switch panels do.

8 Relation to existing notations

CVL is a conservative synthesis: the '•'/'◇' hyperedge notation is Baumgartner and Falk's (2023), extended by the token layer of the neuron-diagram tradition (Baumgartner 2013), the module-box abstraction of the `causal-architect` hypergraph representation, and the derivation-as-deletion discipline of the casebook's proof calculus. It deliberately excludes the arrow vocabulary of DAG/SCM notations: modularity-laden single arrows have no counterpart in the regularity formalism and would break the isomorphism [cf. the demarcation argument of the framework paper].

References

Andreas, Holger, and Mario Günther. 2024. "A Regularity Theory of Causation." *Pacific Philosophical Quarterly* 105 (1): 2–32. https://doi.org/10.1111/papq.12447.

Baumgartner, Michael. 2013. "A Regularity Theoretic Approach to Actual Causation." *Erkenntnis* 78 (S1): 85–109. https://doi.org/10.1007/s10670-013-9438-3.

Baumgartner, Michael, and Christoph Falk. 2023. "Boolean Difference-Making: A Modern Regularity Theory of Causation." *British Journal for the Philosophy of Science* 74 (1): 171–97. https://doi.org/10.1093/bjps/axz047.

Baumgartner, Michael, and Gerd Graßhoff. 2004. *Kausalität und kausales Schliessen. Eine Einführung mit interaktiven Übungen*. Bern Studies in the History; Philosophy of Science.